APEX Success and Error Codes Guide

Version: 2.2 **Date:** 2025-10-09 **Author:** Mark Andrew Ray-Smith Cityline Ltd

Overview

APEX Rules Engine supports **success codes** and **error codes** that allow you to attach business-specific codes to rule and enrichment outcomes. These codes can be:

- **Constants** (e.g., `"OPS-TRADE-0000"`, `"ERR001-5300"`)
- **SpEL expressions** (e.g., `"#amount > 1000 ? 'HIGH_VALUE' : 'NORMAL_VALUE'"`)

Additionally, the `map-to-field` keyword enables you to:

- **Write codes back to the dataset** being validated or enriched
- **Create arrays of error codes** for comprehensive error reporting (e.g., `["ERR001-5300", "ENR005-3750"]`)
- **Add validation metadata** (timestamps, status, messages) directly to your data
- **Enable downstream processing** by making validation results part of the dataset

This makes your datasets **self-contained** and ready for persistence, logging, or integration with external systems.

Key Features

Attach codes to outcomes - Map rule/enrichment results to external system codes (e.g., `ERR001-5300`) **Constant or dynamic codes** - Use fixed strings or SpEL expressions **Write back to dataset** - Validation results become part of the data being processed **Multiple error codes** - Create arrays of codes for comprehensive error reporting **Field mapping** - Write codes and computed values to dataset fields **Chained processing** - Downstream rules can reference mapped fields **Self-contained datasets** - Data includes validation status, codes, timestamps, and metadata **Backward compatible** - All keywords are optional

---

YAML Keywords

For Rules and Enrichments

Keyword	Type	Description	Example
`success-code`	String	Code evaluated when rule matches or enrichment succeeds	`"OPS-TRADE-0000"` or `"#amount > 1000 ? 'HIGH' : 'LOW'"`
`error-code`	String	Code evaluated when rule fails or enrichment fails	`"ERR001-5300"` or `"#errorType"`
`map-to-field`	String or List	SpEL expressions to write values to dataset fields (supports arrays via SpEL)	`"status = #success_code"` or `"codes = new String[]{'ERR001-5300', 'ENR005-3750'}"`

Note: While `success-code` and `error-code` are single String values, you can use `map-to-field` with SpEL to create arrays of codes for scenarios requiring multiple operations codes.

Basic Examples

Example 1: Trade Processing Operations Codes

This example demonstrates how to use **operational error codes** that integrate with external trade processing systems. These codes follow standard operations system conventions (e.g., `ERR001-5300`, `ENR005-3750`).

Example 1a: Single Operations Code

```
rules:
  - id: "trade-settlement-validation"
    name: "Trade Settlement Validation"
    condition: "#settlementDate != null && #settlementAmount > 0"
    message: "Trade settlement validation"
    severity: "ERROR"
    success-code: "OPS-SETTLE-0000"      # Operations success code
    error-code: "ERR001-5300"            # Operations error code for settlement
failure
```

Behavior:

- When condition is **TRUE**: `RuleResult.getSuccessCode()` returns `"OPS-SETTLE-0000"`
- When condition is **FALSE**: `RuleResult.getErrorCode()` returns `"ERR001-5300"`

Use Case: Integration with trade operations systems that require specific error codes for downstream processing, reporting, and reconciliation.

Example 1b: Multiple Operations Codes Using Field Mapping

To assign **multiple error codes** (e.g., `["ERR001-5300", "ENR005-3750"]`), use `map-to-field` to write an array to the dataset:

```
rules:
  - id: "trade-validation-multi-code"
    name: "Trade Validation with Multiple Error Codes"
    condition: "#tradeId != null && #amount > 0 && #currency != null"
    message: "Comprehensive trade validation"
    severity: "ERROR"
    success-code: "OPS-TRADE-0000"
    error-code: "ERR001-5300"
    map-to-field:
      - "primaryErrorCode = #error_code"
      - "allErrorCodes = new String[]{'ERR001-5300', 'ENR005-3750'}"
```

```
      - "operationsCategory = 'TRADE_SETTLEMENT'"
      - "requiresManualReview = true"
```

Behavior:

- When condition is **FALSE**:
  - `primaryErrorCode = "ERR001-5300"` (single code for primary error)
  - `allErrorCodes = ["ERR001-5300", "ENR005-3750"]` (array of all related error codes)
  - Downstream systems can access both single and multiple error codes

Java API Access:

```
RuleResult result = engine.executeRule(rule, data);
String primaryCode = result.getErrorCode();  // "ERR001-5300"
String[] allCodes = (String[]) result.getEnrichedData().get("allErrorCodes");
// allCodes = ["ERR001-5300", "ENR005-3750"]
```

Example 1d: Writing Error Codes Back to the Dataset

Use Case: Write validation results directly back to the trade object being validated, so the enriched dataset contains the validation status and error codes.

```
rules:
  - id: "trade-validation-with-dataset-update"
    name: "Trade Validation - Update Dataset"
    condition: "#tradeId != null && #amount > 0 && #currency != null"
    message: "Trade validation with dataset update"
    severity: "ERROR"
    success-code: "OPS-TRADE-0000"
    error-code: "ERR001-5300"
    map-to-field:
      - "validationStatus = #success_code != null ? 'VALID' : 'INVALID'"
      - "validationCode = #success_code != null ? #success_code : #error_code"
      - "validationErrorCodes = #error_code != null ? new String[]{'ERR001-5300',
'ENR005-3750'} : null"
      - "validationTimestamp = T(java.time.Instant).now()"
      - "validationMessage = #message"
      - "requiresReview = #error_code != null"
```

Behavior:

When validation **succeeds** (condition = TRUE):

```
// Input dataset
Map<String, Object> trade = new HashMap<>();
trade.put("tradeId", "TRD-001");
```

```
trade.put("amount", 1000000);
trade.put("currency", "USD");

// After rule execution, the SAME dataset is enriched with validation results
RuleResult result = engine.executeRule(rule, trade);

// The enriched data is written back to the dataset
Map<String, Object> enrichedData = result.getEnrichedData();
enrichedData.get("validationStatus");      // "VALID"
enrichedData.get("validationCode");        // "OPS-TRADE-0000"
enrichedData.get("validationErrorCodes");  // null
enrichedData.get("requiresReview");        // false
```

When validation **fails** (condition = FALSE):

```
// Input dataset with missing currency
Map<String, Object> trade = new HashMap<>();
trade.put("tradeId", "TRD-002");
trade.put("amount", 1000000);
trade.put("currency", null);  // Missing!

// After rule execution
RuleResult result = engine.executeRule(rule, trade);

// The enriched data contains error information
Map<String, Object> enrichedData = result.getEnrichedData();
enrichedData.get("validationStatus");      // "INVALID"
enrichedData.get("validationCode");        // "ERR001-5300"
enrichedData.get("validationErrorCodes");  // ["ERR001-5300", "ENR005-3750"]
enrichedData.get("requiresReview");        // true
```

Benefits:

- **Self-contained dataset:** All validation results are part of the dataset
- **Downstream processing:** Subsequent rules can reference validation status
- **Audit trail:** Validation timestamp and messages are preserved
- **Operations integration:** Error codes are embedded in the data for external systems
- **Conditional logic:** Downstream rules can check `#validationStatus == 'VALID'`

Example 1c: Dynamic Operations Codes Based on Error Type

```
rules:
  - id: "trade-validation-dynamic-codes"
    name: "Trade Validation with Dynamic Error Codes"
    condition: "#tradeId != null && #amount > 0"
    message: "Trade validation with context-aware error codes"
    severity: "ERROR"
```

```
    success-code: "OPS-TRADE-0000"
    error-code: "#tradeId == null ? 'ERR001-5300' : 'ERR001-5301'"
    map-to-field:
      - "errorCode = #error_code"
      - "errorCodes = #tradeId == null ? new String[]{'ERR001-5300', 'ERR002-
1100'} : new String[]{'ERR001-5301', 'ERR003-2200'}"
      - "errorCategory = #tradeId == null ? 'MISSING_TRADE_ID' : 'INVALID_AMOUNT'"
```

Behavior:

- When `tradeId == null`:
  - `errorCode = "ERR001-5300"`
  - `errorCodes = ["ERR001-5300", "ERR002-1100"]`
  - `errorCategory = "MISSING_TRADE_ID"`
- When `amount <= 0`:
  - `errorCode = "ERR001-5301"`
  - `errorCodes = ["ERR001-5301", "ERR003-2200"]`
  - `errorCategory = "INVALID_AMOUNT"`

Use Case: Different error scenarios require different sets of operations codes for proper routing and handling in downstream systems.

Example 2: Dynamic Success Code with SpEL

```
rules:
  - id: "risk-assessment"
    name: "Risk Assessment"
    condition: "#amount > 0"
    success-code: "#amount > 100000 ? 'HIGH_RISK' : 'LOW_RISK'"
    error-code: "INVALID_AMOUNT"
```

Behavior:

- When condition is TRUE and `amount = 150000`: `successCode = "HIGH_RISK"`
- When condition is TRUE and `amount = 50000`: `successCode = "LOW_RISK"`
- When condition is FALSE: `errorCode = "INVALID_AMOUNT"`

Example 3: Field Mapping (Single Field)

```
rules:
  - id: "validation-rule"
    condition: "#amount > 0"
    success-code: "VALID"
    error-code: "INVALID"
    map-to-field: "validationStatus = #success_code"
```

Behavior:

- When rule matches: Writes `validationStatus = "VALID"` to the dataset
- When rule fails: Writes `validationStatus = "INVALID"` to the dataset
- Downstream rules can reference `#validationStatus` in their conditions

Example 4: Field Mapping (Multiple Fields)

```
rules:
  - id: "audit-trail"
    condition: "#amount > 0"
    success-code: "TX_APPROVED"
    error-code: "TX_REJECTED"
    map-to-field:
      - "auditCode = #success_code"
      - "auditTimestamp = T(java.time.Instant).now()"
      - "auditSeverity = #severity"
      - "auditMessage = #message"
```

Behavior:

- Writes multiple fields to the dataset in a single rule evaluation
- All mapped fields become available to downstream rules via SpEL

Advanced Use Cases

Use Case 1: Chained Rules Using Field Mapping

Use Case: Use the output of one rule as input to another rule, creating a validation pipeline.

```
rules:
  # First rule: Validate amount and set status
  - id: "amount-validation"
    condition: "#amount > 0"
    success-code: "AMOUNT_VALID"
    error-code: "ERR-AMOUNT-8001"
    map-to-field: "amountStatus = #success_code != null ? #success_code :
#error_code"

  # Second rule: Use mapped field from first rule
  - id: "currency-validation"
    condition: "#amountStatus == 'AMOUNT_VALID' && #currency != null"
    success-code: "FULL_VALIDATION_PASSED"
    error-code: "ERR-CURRENCY-8002"
    map-to-field: "validationResult = #success_code != null ? #success_code :
#error_code"
```

Processing Flow:

1. First rule evaluates, writes `amountStatus` to dataset
2. Second rule references `#amountStatus` in its condition
3. Second rule writes `validationResult` to dataset

Benefits:

- Simple sequential validation
- Each rule builds on previous results
- Clear validation pipeline

Use Case 2: Dynamic Risk Scoring

Use Case: Calculate risk scores and assign risk levels based on trade amount.

```
rules:
  - id: "risk-scoring"
    condition: "#amount > 0"
    success-code: "#amount > 100000 ? 'HIGH_RISK' : 'LOW_RISK'"
    map-to-field:
      - "riskLevel = #success_code"
      - "riskScore = #amount > 100000 ? 100 : 50"
      - "requiresApproval = #amount > 50000"
```

Benefits:

- Dynamic code assignment based on business logic
- Multiple related fields calculated together
- Approval flags set automatically

Use Case 3: Conditional Error Code Assignment Based on Trade Attributes

Use Case: Assign different error codes based on trade type, notional amount, and counterparty risk rating to route trades to appropriate operations teams.

```
rules:
  - id: "trade-routing-validation"
    name: "Trade Routing with Dynamic Error Codes"
    condition: "#tradeType != null && #notional > 0 && #counterpartyRating !=
null"
    message: "Trade routing validation"
    severity: "ERROR"
    success-code: "OPS-ROUTE-0000"
    error-code: |
      #tradeType == 'EXOTIC_OPTION' ? 'ERR-EXOTIC-5300' :
      #notional > 10000000 ? 'ERR-LARGE-5301' :
      #counterpartyRating == 'HIGH_RISK' ? 'ERR-RISK-5302' :
```

```
      'ERR-GENERAL-5399'
    map-to-field:
      - "routingCode = #success_code != null ? #success_code : #error_code"
      - "routingTeam = #error_code == 'ERR-EXOTIC-5300' ? 'EXOTIC_DESK' :
#error_code == 'ERR-LARGE-5301' ? 'LARGE_TRADE_DESK' : #error_code == 'ERR-RISK-
5302' ? 'CREDIT_RISK_TEAM' : 'GENERAL_OPS'"
      - "priority = #notional > 10000000 || #counterpartyRating == 'HIGH_RISK' ?
'HIGH' : 'NORMAL'"
      - "requiresSeniorApproval = #notional > 50000000"
```

Benefits:

- Different error codes route to different operations teams
- Priority automatically assigned based on trade characteristics
- Senior approval flag set for large trades

Use Case 4: Time-Based Error Code Assignment

Use Case: Assign different error codes based on trade settlement date proximity to handle urgent vs. standard processing.

```
rules:
  - id: "settlement-urgency-validation"
    name: "Settlement Urgency Validation"
    condition: "#settlementDate != null && #tradeDate != null"
    message: "Settlement date validation with urgency classification"
    severity: "WARNING"
    success-code: "SETTLE-VALID-0000"
    error-code: |
      T(java.time.temporal.ChronoUnit).DAYS.between(
        T(java.time.LocalDate).parse(#tradeDate),
        T(java.time.LocalDate).parse(#settlementDate)
      ) <= 1 ? 'ERR-URGENT-7001' : 'ERR-STANDARD-7002'
    map-to-field:
      - "settlementUrgency = #error_code == 'ERR-URGENT-7001' ? 'URGENT' :
'STANDARD'"
      - "settlementErrorCode = #error_code"
      - "daysToSettle =
T(java.time.temporal.ChronoUnit).DAYS.between(T(java.time.LocalDate).parse(#tradeD
ate), T(java.time.LocalDate).parse(#settlementDate))"
      - "requiresExpediting = #settlementUrgency == 'URGENT'"
      - "opsTeam = #settlementUrgency == 'URGENT' ? 'URGENT_SETTLEMENT_TEAM' :
'STANDARD_SETTLEMENT_TEAM'"
```

Benefits:

- Automatic urgency classification based on settlement timeline
- Different error codes for urgent vs. standard processing
- Automatic team assignment based on urgency

- Calculates days to settlement for reporting
- Uses Java time API for date calculations

Use Case 5: Aggregating Multiple Validation Failures

Use Case: Collect all validation failures across multiple rules and create a comprehensive error report with all error codes.

```
rules:
  # Rule 1: Trade ID validation
  - id: "trade-id-validation"
    condition: "#tradeId != null && #tradeId.length() > 0"
    success-code: "VAL-TRADEID-0000"
    error-code: "ERR-TRADEID-6001"
    map-to-field:
      - "tradeIdValid = #success_code != null"
      - "tradeIdError = #error_code"

  # Rule 2: Counterparty validation
  - id: "counterparty-validation"
    condition: "#counterparty != null && #counterparty.length() > 0"
    success-code: "VAL-CPTY-0000"
    error-code: "ERR-CPTY-6002"
    map-to-field:
      - "counterpartyValid = #success_code != null"
      - "counterpartyError = #error_code"

  # Rule 3: Notional validation
  - id: "notional-validation"
    condition: "#notional != null && #notional > 0"
    success-code: "VAL-NOTIONAL-0000"
    error-code: "ERR-NOTIONAL-6003"
    map-to-field:
      - "notionalValid = #success_code != null"
      - "notionalError = #error_code"

  # Rule 4: Aggregate all errors
  - id: "aggregate-validation-errors"
    condition: "#tradeIdValid && #counterpartyValid && #notionalValid"
    success-code: "TRADE-VALID-0000"
    error-code: "TRADE-INVALID-6999"
    map-to-field:
      - "overallValid = #success_code != null"
      - "overallCode = #success_code != null ? #success_code : #error_code"
      - "allErrorCodes = #success_code != null ? null : new
java.util.ArrayList(java.util.Arrays.asList(#tradeIdError, #counterpartyError,
#notionalError)).stream().filter(e -> e != null).toArray(String[]::new)"
      - "errorCount = #allErrorCodes != null ? #allErrorCodes.length : 0"
      - "validationSummary = #success_code != null ? 'All validations passed' :
'Failed ' + #errorCount + ' validation(s)'"
```

Benefits:

- Collects all validation errors in a single array
- Provides error count and summary message
- Single overall validation status
- Complete audit trail of all validation failures
- Uses Java streams to filter null values

```
rules:
  # First rule: Validate amount and set status
  - id: "amount-validation"
    condition: "#amount > 0"
    success-code: "AMOUNT_VALID"
    error-code: "ERR-AMOUNT-8001"
    map-to-field: "amountStatus = #success_code != null ? #success_code :
#error_code"

  # Second rule: Use mapped field from first rule
  - id: "currency-validation"
    condition: "#amountStatus == 'AMOUNT_VALID' && #currency != null"
    success-code: "FULL_VALIDATION_PASSED"
    error-code: "ERR-CURRENCY-8002"
    map-to-field: "validationResult = #success_code != null ? #success_code :
#error_code"
```

Processing Flow:

1. First rule evaluates, writes `amountStatus` to dataset
2. Second rule references `#amountStatus` in its condition
3. Second rule writes `validationResult` to dataset

Use Case 5: Dynamic Risk Scoring

```
rules:
  - id: "risk-scoring"
    condition: "#amount > 0"
    success-code: "#amount > 100000 ? 'HIGH_RISK' : 'LOW_RISK'"
    map-to-field:
      - "riskLevel = #success_code"
      - "riskScore = #amount > 100000 ? 100 : 50"
      - "requiresApproval = #amount > 50000"
```

Enrichment Examples

Example 5: Enrichment with Success/Error Codes

```yaml
enrichments:
  - id: "customer-lookup"
    type: "lookup-enrichment"
    condition: "#customerId != null"
    success-code: "CUSTOMER_FOUND"
    error-code: "CUSTOMER_NOT_FOUND"
    map-to-field: "lookupStatus = #success_code"
    lookup-config:
      lookup-key: "#customerId"
      lookup-dataset:
        type: "inline"
        data:
          - id: "C001"
            name: "John Doe"
          - id: "C002"
            name: "Jane Smith"
    field-mappings:
      - source-field: "name"
        target-field: "customerName"
```

Behavior:

- When lookup succeeds: `lookupStatus = "CUSTOMER_FOUND"` is written to dataset
- When lookup fails: `lookupStatus = "CUSTOMER_NOT_FOUND"` is written to dataset

Example 6: Dynamic Enrichment Codes

```yaml
enrichments:
  - id: "pricing-enrichment"
    type: "lookup-enrichment"
    success-code: "#impliedVolatility > 30 ? 'HIGH_VOL_PRICING' :
'NORMAL_VOL_PRICING'"
    error-code: "PRICING_DATA_NOT_AVAILABLE"
    map-to-field:
      - "pricingStatus = #success_code"
      - "pricingTimestamp = T(java.time.Instant).now()"
    lookup-config:
      lookup-key: "#symbol"
      lookup-dataset:
        type: "inline"
        data:
          - symbol: "AAPL"
            volatility: 25.5
          - symbol: "TSLA"
            volatility: 45.8
    field-mappings:
      - source-field: "volatility"
        target-field: "impliedVolatility"
```

Available Context Variables in `map-to-field`

When using `map-to-field`, the following variables are available in SpEL expressions:

Variable	Type	Description	Example
`#success_code`	String	The evaluated success code (when rule matches)	`"status = #success_code"`
`#error_code`	String	The evaluated error code (when rule fails)	`"status = #error_code"`
`#message`	String	The rule/enrichment message	`"auditMessage = #message"`
`#severity`	String	The rule severity (ERROR, WARNING, INFO, CRITICAL)	`"auditSeverity = #severity"`
All input fields	Various	Any field from the input dataset	`"riskScore = #amount > 1000 ? 100 : 50"`

Note: Use `#success_code` and `#error_code` (with underscores) in `map-to-field` expressions, not the kebab-case YAML keywords.

Java API Usage

Accessing Codes in RuleResult

```
// Execute rule
RuleResult result = engine.executeRule(rule, data);

// Access success/error codes
String successCode = result.getSuccessCode();  // Non-null when rule matched
String errorCode = result.getErrorCode();      // Non-null when rule failed

// Access enriched data (from map-to-field)
Map<String, Object> enrichedData = result.getEnrichedData();
String validationStatus = (String) enrichedData.get("validationStatus");
```

Example Test Code

```
@Test
public void testSuccessCode() {
    // Load configuration
    var config = yamlLoader.loadFromFile("path/to/config.yaml");
    RulesEngine engine = RulesEngine.fromYamlConfig(config);

    // Get rule
    var rule = engine.getConfiguration().getRuleById("amount-validation");
```

```
    // Execute with matching data
    Map<String, Object> data = new HashMap<>();
    data.put("amount", 150);

    RuleResult result = engine.executeRule(rule, data);

    // Verify success code
    assertTrue(result.isTriggered());
    assertEquals("AMOUNT_VALID", result.getSuccessCode());

    // Verify field mapping
    assertEquals("AMOUNT_VALID",
result.getEnrichedData().get("validationStatus"));
}
```

Best Practices

1. Use Meaningful Code Names

Good:

```
success-code: "TRADE_VALIDATED"
error-code: "TRADE_VALIDATION_FAILED"
```

Bad:

```
success-code: "OK"
error-code: "ERR"
```

---

2. Use Constants for Simple Cases, SpEL for Dynamic Cases

Constant (simple):

```
success-code: "VALID"
```

SpEL (dynamic):

```
success-code: "#amount > 100000 ? 'HIGH_VALUE' : 'NORMAL_VALUE'"
```

---

3. Use `map-to-field` for Chained Processing

When downstream rules need to reference the outcome of earlier rules:

```
rules:
  - id: "step-1"
    condition: "#amount > 0"
    success-code: "STEP_1_PASSED"
    map-to-field: "step1Status = #success_code"  # Write to dataset

  - id: "step-2"
    condition: "#step1Status == 'STEP_1_PASSED'"  # Reference mapped field
    success-code: "STEP_2_PASSED"
```

---

4. Use Descriptive Field Names in Mappings

Good:

```
map-to-field:
  - "validationStatus = #success_code"
  - "validationTimestamp = T(java.time.Instant).now()"
  - "validationSeverity = #severity"
```

Bad:

```
map-to-field:
  - "s = #success_code"
  - "t = T(java.time.Instant).now()"
  - "v = #severity"
```

---

5. Write Validation Results Back to Dataset

Best Practice: Use `map-to-field` to write validation/enrichment results directly back to the dataset being processed. This makes the dataset self-contained and ready for persistence or downstream processing.

Good - Write results to dataset:

```
rules:
  - id: "trade-validation"
    condition: "#tradeId != null && #amount > 0"
    success-code: "OPS-TRADE-0000"
    error-code: "ERR001-5300"
    map-to-field:
      - "validationStatus = #success_code != null ? 'VALID' : 'INVALID'"
```

```
      - "validationCode = #success_code != null ? #success_code : #error_code"
      - "validationTimestamp = T(java.time.Instant).now()"
      - "validatedBy = 'APEX_ENGINE'"
```

Benefits:

- Dataset contains complete validation context
- Can be persisted to database with audit trail
- Downstream systems receive full validation metadata
- No need to maintain separate validation results

Less Ideal - Results only in RuleResult:

```
rules:
  - id: "trade-validation"
    condition: "#tradeId != null && #amount > 0"
    success-code: "OPS-TRADE-0000"
    error-code: "ERR001-5300"
    # No map-to-field - results only accessible via RuleResult API
```

Limitations:

- Validation results not embedded in dataset
- Requires separate handling of RuleResult
- Dataset and validation results are disconnected
- More complex to persist or pass to downstream systems

---

6. Handle Invalid SpEL Gracefully

APEX logs warnings for invalid SpEL expressions but does not break execution:

```
# Invalid SpEL expression
success-code: "#invalidField.nonExistentMethod()"
```

Behavior:

- Warning is logged
- `successCode` is set to `null`
- Rule evaluation continues normally

Common Patterns

Pattern 1: Trade Operations System Integration

Use Case: Integrate APEX validation results with external trade operations systems that require specific error codes for routing, reporting, and reconciliation.

```yaml
rules:
  - id: "trade-booking-validation"
    name: "Trade Booking Validation"
    condition: "#tradeId != null && #counterparty != null && #notional > 0"
    message: "Trade booking validation for operations system"
    severity: "ERROR"
    success-code: "OPS-BOOK-0000"
    error-code: "ERR001-5300"
    map-to-field:
      - "opsStatusCode = #success_code != null ? #success_code : #error_code"
      - "opsErrorCodes = #error_code != null ? new String[]{'ERR001-5300',
'ENR005-3750'} : null"
      - "opsCategory = 'TRADE_BOOKING'"
      - "opsTimestamp = T(java.time.Instant).now()"
      - "requiresOpsReview = #error_code != null"

enrichments:
  - id: "trade-enrichment-with-ops-codes"
    type: "lookup-enrichment"
    success-code: "ENR-PRICING-0000"
    error-code: "ENR005-3750"
    map-to-field:
      - "pricingStatusCode = #success_code != null ? #success_code : #error_code"
      - "pricingErrorCodes = #error_code != null ? new String[]{'ENR005-3750',
'ENR006-4100'} : null"
    lookup-config:
      lookup-key: "#symbol"
      lookup-dataset:
        type: "inline"
        data:
          - symbol: "AAPL"
            price: 175.50
    field-mappings:
      - source-field: "price"
        target-field: "marketPrice"
```

Benefits:

- Standardized error codes across all systems
- Single error code for primary routing (`opsStatusCode`)
- Multiple error codes for comprehensive reporting (`opsErrorCodes`)
- Automatic flagging for manual review (`requiresOpsReview`)
- Audit trail with timestamps

Pattern 2: Business Error Code Mapping

```
rules:
  - id: "compliance-check"
    condition: "#trade.isCompliant()"
    success-code: "COMPLIANT"
    error-code: "#trade.getComplianceFailureCode()"
    map-to-field: "complianceStatus = #success_code"
```

Pattern 3: Audit Trail with Timestamps

```
rules:
  - id: "audit-trail"
    condition: "#transaction.isValid()"
    success-code: "TX_APPROVED"
    error-code: "TX_REJECTED"
    map-to-field:
      - "auditCode = #success_code"
      - "auditTimestamp = T(java.time.Instant).now()"
      - "auditMessage = #message"
      - "auditSeverity = #severity"
```

Pattern 4: Writing Validation Results Back to Dataset

Use Case: Enrich the original dataset with validation results so it becomes self-contained and can be persisted, logged, or sent to downstream systems with all validation metadata embedded.

```
rules:
  - id: "trade-validation-enrich-dataset"
    name: "Trade Validation - Enrich Original Dataset"
    condition: "#tradeId != null && #notional > 0 && #counterparty != null"
    message: "Trade validation for booking"
    severity: "ERROR"
    success-code: "OPS-BOOK-0000"
    error-code: "ERR001-5300"
    map-to-field:
      # Write validation status back to dataset
      - "trade.validationStatus = #success_code != null ? 'APPROVED' : 'REJECTED'"
      - "trade.validationCode = #success_code != null ? #success_code :
#error_code"
      - "trade.validationErrors = #error_code != null ? new String[]{'ERR001-
5300', 'ENR005-3750'} : null"
      - "trade.validationTimestamp = T(java.time.Instant).now()"
      - "trade.validatedBy = 'APEX_RULES_ENGINE'"
      - "trade.requiresManualReview = #error_code != null"
      - "trade.bookingEligible = #success_code != null"

enrichments:
```

```
  - id: "pricing-enrichment-with-status"
    type: "lookup-enrichment"
    success-code: "ENR-PRICING-0000"
    error-code: "ENR005-3750"
    map-to-field:
      # Write enrichment status back to dataset
      - "trade.pricingStatus = #success_code != null ? 'PRICED' :
'PRICING_FAILED'"
      - "trade.pricingCode = #success_code != null ? #success_code : #error_code"
      - "trade.pricingTimestamp = T(java.time.Instant).now()"
      - "trade.pricingSource = 'MARKET_DATA_LOOKUP'"
    lookup-config:
      lookup-key: "#symbol"
      lookup-dataset:
        type: "inline"
        data:
          - symbol: "AAPL"
            price: 175.50
    field-mappings:
      - source-field: "price"
        target-field: "trade.marketPrice"
```

Java Example:

```
// Original trade object
Map<String, Object> trade = new HashMap<>();
trade.put("tradeId", "TRD-001");
trade.put("notional", 1000000);
trade.put("counterparty", "BANK-A");
trade.put("symbol", "AAPL");

// Execute validation and enrichment
RuleResult result = engine.executeRule(rule, trade);

// The enriched data now contains all validation metadata
Map<String, Object> enrichedData = result.getEnrichedData();

// Access nested trade object with validation results
Map<String, Object> enrichedTrade = (Map<String, Object>)
enrichedData.get("trade");
enrichedTrade.get("validationStatus");      // "APPROVED"
enrichedTrade.get("validationCode");        // "OPS-BOOK-0000"
enrichedTrade.get("validationTimestamp");   // 2025-10-09T14:30:00Z
enrichedTrade.get("bookingEligible");       // true
enrichedTrade.get("pricingStatus");         // "PRICED"
enrichedTrade.get("marketPrice");           // 175.50

// The enriched trade object can now be:
// - Persisted to database with all validation metadata
// - Sent to downstream systems (booking, settlement, reporting)
// - Logged for audit trail
// - Used in subsequent processing stages
```

Benefits:

- **Self-contained data:** All validation/enrichment results embedded in the dataset
- **Persistence-ready:** Can be saved to database with full audit trail
- **Downstream integration:** External systems receive complete validation context
- **Audit compliance:** Timestamp, validator, and error codes are preserved
- **Conditional routing:** Downstream systems can route based on `validationStatus`

Pattern 5: Multi-Stage Validation Pipeline

```
rules:
  # Stage 1: Amount validation
  - id: "amount-check"
    condition: "#amount > 0"
    success-code: "AMOUNT_VALID"
    error-code: "ERR-AMOUNT-001"
    map-to-field:
      - "amountStatus = #success_code != null ? #success_code : #error_code"
      - "amountValidated = #success_code != null"

  # Stage 2: Currency validation (depends on Stage 1)
  - id: "currency-check"
    condition: "#amountValidated == true && #currency != null"
    success-code: "CURRENCY_VALID"
    error-code: "ERR-CURRENCY-001"
    map-to-field:
      - "currencyStatus = #success_code != null ? #success_code : #error_code"
      - "currencyValidated = #success_code != null"

  # Stage 3: Final validation (depends on Stage 1 and 2)
  - id: "final-check"
    condition: "#amountValidated == true && #currencyValidated == true"
    success-code: "VALIDATION_COMPLETE"
    error-code: "VALIDATION_INCOMPLETE"
    map-to-field:
      - "finalStatus = #success_code != null ? #success_code : #error_code"
      - "allValidationCodes = new String[]{#amountStatus, #currencyStatus,
#finalStatus}"
      - "readyForBooking = #success_code != null"
```

Troubleshooting

Issue 1: Success Code is Null

Problem: `result.getSuccessCode()` returns `null` even though rule matched.

Solution: Check that `success-code` is defined in YAML:

```
rules:
  - id: "my-rule"
    condition: "#amount > 0"
    success-code: "VALID"  # Add this
```

Issue 2: Field Mapping Not Working

Problem: Mapped field is not available in downstream rules.

Solution: Verify the mapping syntax uses `=` and correct variable names:

```
map-to-field: "status = #success_code"  # Correct
# NOT: "status: #success_code"          # Wrong syntax
```

Issue 3: SpEL Expression Error

Problem: SpEL expression in code throws error.

Solution: Check expression syntax and available variables:

```
# Correct - uses available field
success-code: "#amount > 1000 ? 'HIGH' : 'LOW'"

# Wrong - references non-existent field
success-code: "#invalidField > 1000 ? 'HIGH' : 'LOW'"
```

Issue 4: Mapped Field Not Available in Downstream Rule

Problem: Downstream rule cannot reference field mapped by earlier rule.

Solution: Ensure the earlier rule completes before the downstream rule executes. In APEX, rules execute in document order, so place the mapping rule before the consuming rule:

```
rules:
  # Correct order
  - id: "step-1"
    map-to-field: "status = #success_code"

  - id: "step-2"
    condition: "#status == 'VALID'"  # Can reference status
```

Testing Examples

See the following test files for complete working examples:

- **Basic codes:** `apex-demo/src/test/java/dev/mars/apex/demo/codes/SuccessErrorCodesValidation.java`
- **Field mapping:** `apex-demo/src/test/java/dev/mars/apex/demo/codes/FieldMappingValidation.java`
- **Enrichment codes:** `apex-demo/src/test/java/dev/mars/apex/demo/codes/EnrichmentCodesValidation.java`
- **Trade validation demo:** `apex-demo/src/test/java/dev/mars/apex/demo/codes/TradeValidationCodesDemo.java`

Implementation Status

Phase 1: YAML Configuration Layer - Complete **Phase 2: Core Model Layer** - Complete **Phase 3: Result Layer** - Complete **Phase 4: Evaluation and Mapping Logic** - Complete **Phase 5: Comprehensive Testing** - Complete

All features are fully implemented and tested.

Summary

success-code and **error-code** attach business codes to rule/enrichment outcomes **map-to-field** writes codes and computed values to dataset fields Supports both **constant strings** and **SpEL expressions** Enables **chained processing** where downstream rules reference mapped fields **Fully backward compatible** - all keywords are optional **Comprehensive test coverage** in apex-demo module

---

Related Documentation

- **Design Document:** `docs/design/ERROR_SUCCESS_CODES_DESIGN.md`
- **YAML Reference:** `docs/APEX_YAML_REFERENCE.md`
- **SpEL Guide:** `docs/APEX_SPEL_GUIDE.md`
- **Error Handling Guide:** `docs/APEX_ERROR_HANDLING_GUIDE.md`
- **User Guide:** `docs/APEX_RULES_ENGINE_USER_GUIDE.md`